CODEBASE BLUEPRINT — READ AGAINST SHARED/PROTOCOL.MD & PLAN.MD

BRUTALTANK, MAPPED

Every box below reflects the code as it stands on the `master` branch (commit `1f54e78`), not the aspirational shape in `PLAN.md` — the two have drifted in a couple of places, called out where it matters. Updated this pass to cover the two biggest additions since the last read: a full **AI bot player** subsystem, and a weapon **physics/rendering audit** that retuned the power/wind curves and fixed two real trajectory-rendering bugs.

+ BOTS SUBSYSTEM (FIG. 3)

+ PHYSICS/RENDERING FIXES (FIG. 6)

Source: PLAN.md · shared/protocol.md · client/src/** · server/src/main/java/com/brutaltank/**
read 2026-08-25, commit 1f54e78

FIG. 1

SYSTEM OVERVIEW

`client ⇔ server`

Two independently-buildable modules, connected only by the JSON envelope contract in `shared/protocol.md`. Bots (fig. 3) don't change this picture at all — they're a purely server-side addition with no new wire messages, driven through `Match`'s existing methods.

Client module (Svelte/TS) · Server module (Java) · Plain domain/data class · Broadcast event / flagged drift

CLIENT — Svelte + TS, Vite :5173

App.svelte

Screen router — reads `matchStore.status` / `lobbyStore.matchId`.

lib/stores/*

matchStore · lobbyStore · sessionStore · connectionStore · aimStore · weaponSelectStore · shotAnimationStore. Each independently subscribes `wsClient.onMessage()` and reduces one slice of protocol events; matchStore is authoritative.

No client-side prediction — the UI waits for ShotResolved before animating.

lib/net/ + lib/protocol/

wsClient (connect/backoff/reconnect/ping), lobbyActions, envelope.ts, types.ts

lib/game/render/ + lib/game/input/

Canvas draw fns (terrain/tank/projectile) · aim/fire/shop senders

⇄

`{ type, v, requestId, payload }`
`shared/protocol.md` — hand-maintained, no codegen

SERVER — Java + Undertow, :6154/ws

BrutalTankServer

Undertow bootstrap, envelope dispatch, per-connection idle-reap (90s) paired with client Ping every 20s.

LobbyManager

Thin router: CreateMatch / JoinMatch / SetReady / Rejoin / Leave / PlayAgain — plus spawning bots into a new match (fig. 3).

MatchRegistry

matchId → Match, ConcurrentHashMap (only cross-match shared state)

Match

One match's whole lifecycle — WAITING → IN_PROGRESS ⇄ SHOP → COMPLETE (→ WAITING again via PlayAgain). Every mutator is `synchronized(this)`; broadcasts results itself. Optionally owns one BotController.

see fig. 2 for its full neighborhood, fig. 3 for the bot side

FIG. 2

SERVER DOMAIN MODEL

`Match as hub`

Everything server-side either *is* Match's state or gets called by it — now including one optional new dependency, `BotController` (fig. 3 has its own detail view). Read top-to-bottom: plumbing that constructs/routes to a Match, Match itself, then everything it owns or calls into.

ROUTES TO A MATCH

BrutalTankServer → LobbyManager → MatchRegistry
Undertow bootstrap → thin CreateMatch/JoinMatch/SetReady/Rejoin/Leave/PlayAgain router → `matchId → Match` map. *LobbyManager also spawns bots into a fresh match here — fig. 3.*

MatchConfig
`record(maxRounds, maxPlayers, botCount, botDifficulty)` — the last two are new: resolved from `CreateMatch.matchConfig`, clamped so `botCount` always leaves room for the creator.

PlayerSession
1 per real WS channel; identifies the caller for every human request. Bots have none — see fig. 3.

▾ configures / identifies

Match

`status · players`
`terrain, turnOrder`
`wind{strength,dirSign}`
`shopStockRemaining`
`botController (nullable)`

`addPlayer()` `addBot()` `setReady()` `fire()` `purchase()` `shopContinue()` `updateAim()` `rematch()` `buildStateSync()`

Every mutator: `synchronized(this)`. `turnToken`/`shopToken` guard timer races against a same-instant Fire/Purchase — and now against a bot's own scheduled turn/shop action too (fig. 3 uses the identical guard).

▾ owns / calls

Player → Tank
playerId, cash, loadout, activeShieldId → x, y, health, alive. `MatchPlayer` wraps this per-match with `playerToken/sink/isBot`.
`owns`

Terrain → TerrainGenerator
`heights: int[1600]`. `applyCrater()` (additive blast) + new `carveTunnelSegment()` (max-based, path-following — fig. 6).
`owns / creates`

WeaponDef · ShieldDef
Static rosters. `ProjectileSim.POWER_SCALE` 12.0→9.0, `WIND_ACCEL_PER_STRENGTH` 4.0→2.5 — fig. 6.
`byId() lookup`

ProjectileSim · DamageCalculator
`simulate()` / `resolve()` — static, stateless. Every bot shot reuses the exact same `simulate()` a human's Fire does.
`calls (static)`

BotController (new)
Lazily created on the first `addBot()`. Hooked into `beginTurn()`/`openShop()`. Full detail in fig. 3.
`owned (lazy) / drives`

Payloads
Records mirroring shared/protocol.md, one shared ObjectMapper. Every per-player DTO (`LobbyPlayerDto`, `StartedPlayerDto`, `PlayerDto`) now carries `isBot`.
`builds → Envelopes.send()`

DRIFT (unchanged since the last read) — PLAN.md §2.2 specifies a queue-based `MatchActor`. The shipped code stays on the simpler `synchronized(this)` + token-guard path documented in Match.java's own javadoc. Bots don't change this call: `BotController` calls straight into the same synchronized methods a WebSocket handler would, no separate concurrency story.

FIG. 3 — NEW THIS PASS

BOTS SUBSYSTEM

`server-only, no new wire messages`

A bot is a `MatchPlayer` with `sink=null` — `broadcast()` already null/open-checks every sink, so this needed no change there. It stays `connected=true` forever (counts toward turn order/ready checks like a human) but is structurally unreachable by the disconnect/reap machinery, since nothing ever calls `handleDisconnect` on it. Everything below lives in `server/…/match/`: `Difficulty`, `BotProfile`, `BotAimPlanner`, `BotShopPlanner`, `BotController`.

SETUP — ONCE, AT CREATEMATCH

MenuScreen
Bot count (0-6) + difficulty selector

→

CreateMatch.matchConfig
`botCount`, `botDifficulty`

→

LobbyManager.spawnBots()
Loop `botCount` times, names from a fixed pool

→

Match.addBot()
`addPlayer(name, null)` → mark isBot → lazily create BotController → `setReady(true)`

TURN LOOP — EVERY TURNSTARTED

Match.beginTurn()
hook: `botController.onTurnStarted(id, turnToken)`

→

1.5-4.5s "thinking" delay
scheduler.schedule(), so bots don't act instantly

→

BotAimPlanner.plan()
Pure. Grid-searches ProjectileSim for a hit, degrades it per skill — fig. 6

→

Match.updateAim()
Broadcasts PlayerAiming — turret visibly snaps *(new fix, fig. 6)*

→

~500ms pause
then re-checks `isTurnTokenCurrent`

→

Match.fire()
Identical call a human's Fire message makes

SHOP LOOP — EVERY SHOPOPENED

Match.openShop()
hook: `botController.onShopOpened(shopToken)`, once per bot

→

0.8-3.3s delay, per bot
staggered so bots don't all act at once

→

BotShopPlanner.plan()
Pure. Spends a profile-scaled cash fraction, value-weighted or impulsive

→

Match.purchase() ×N
Same shared stock pool a human buys from

→

Match.shopContinue()
Required — a bot is never auto-skipped, it stays "connected"

Difficulty tiers — BotProfile.forDifficulty

TIER	ANGLE ERR	POWER ERR	WIND-AWARE	WILD MISS	MONEY SENSE
EASY	12-25°	6-12	never	~12%	low, reckless spend
MEDIUM	5-12°	2-5	50/50 per shot	~5%	moderate
HARD	1-5°	0.5-2	always	~1%	efficient, keeps reserve
MIXED	resolves to a random EASY/MEDIUM/HARD *per bot* — not one blended tier				

Safety guards

- `isTurnTokenCurrent/isShopTokenCurrent` — same staleness pattern as the human timeout race
- Grid search rejects any candidate that would wrap the 1600-unit map — fig. 6
- Test-only `setBotThinkDelayRangeMs()` collapses all the delays above for fast integration tests

Fig. 3 — no fake WebSocket, no client-side bot simulation: a bot's entire footprint is one extra `MatchPlayer` and these five small files calling straight into Match's existing public methods.

FIG. 4

CLIENT COMPONENT & STORE MAP

Svelte tree

One router, five screens, one canvas. Every store independently subscribes to the same wsClient message stream and reduces its own slice — there's no central action dispatcher. Two small additions this pass, both purely cosmetic (`isBot` just flows through the same envelopes every other field already does).

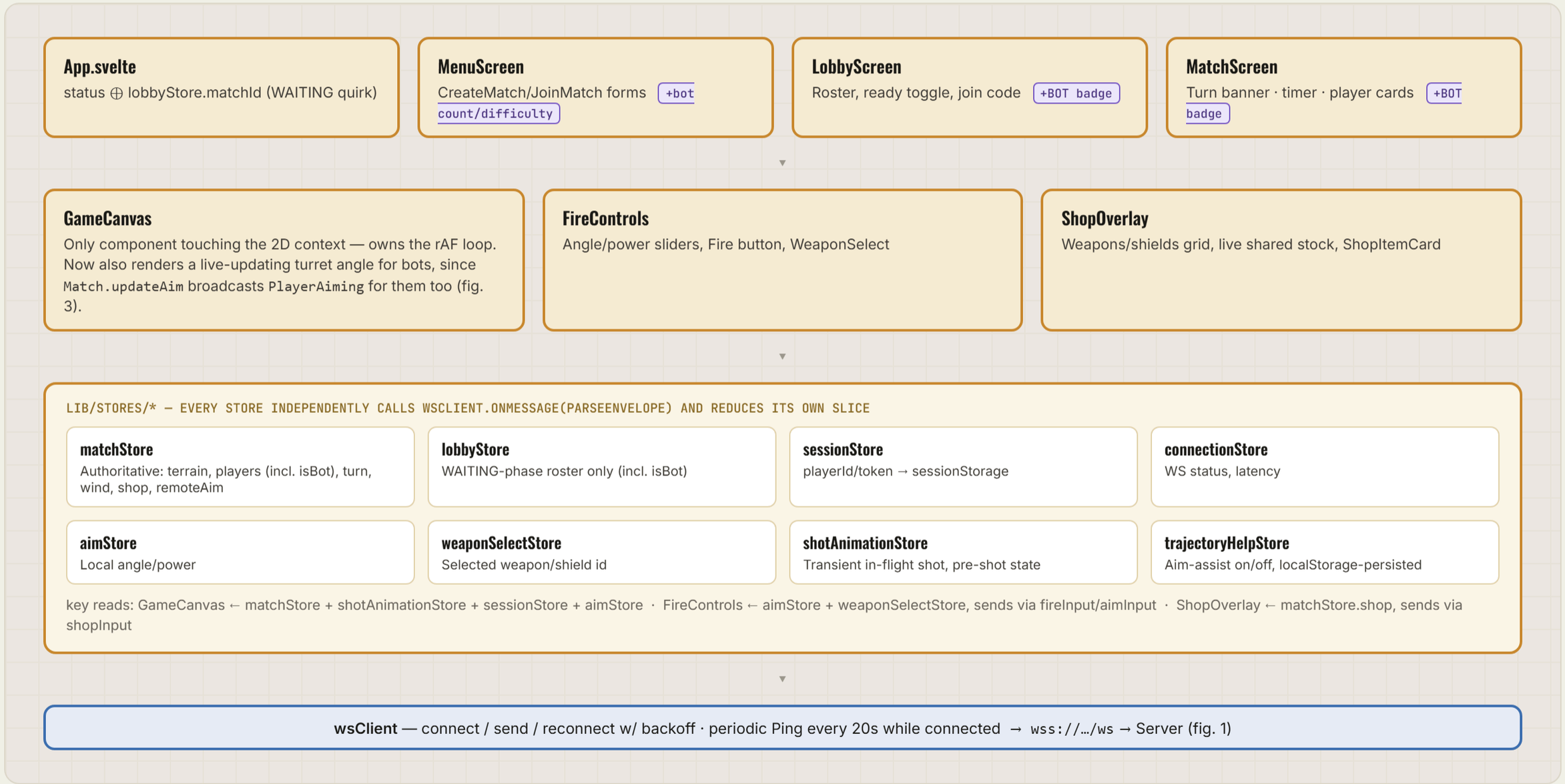

Fig. 4 — no store owns another; they're siblings that happen to agree on shape because they all read the same envelopes. Bots never touch this whole diagram directly — every `isBot`/aim/turn update a bot causes arrives as an ordinary broadcast, same code path as a human's.

FIG. 5

MATCH LIFECYCLE & MESSAGE FLOW

the actual loop

What a player experiences, and the message that drives each transition. Bots drive themselves through this *exact* loop — no shortcuts, no separate path (fig. 3). Two real fixes since the last read: the shop timeout is a 600s safety net, not the 30s shown before; and a disconnect during SHOP is now handled instead of silently ignored.

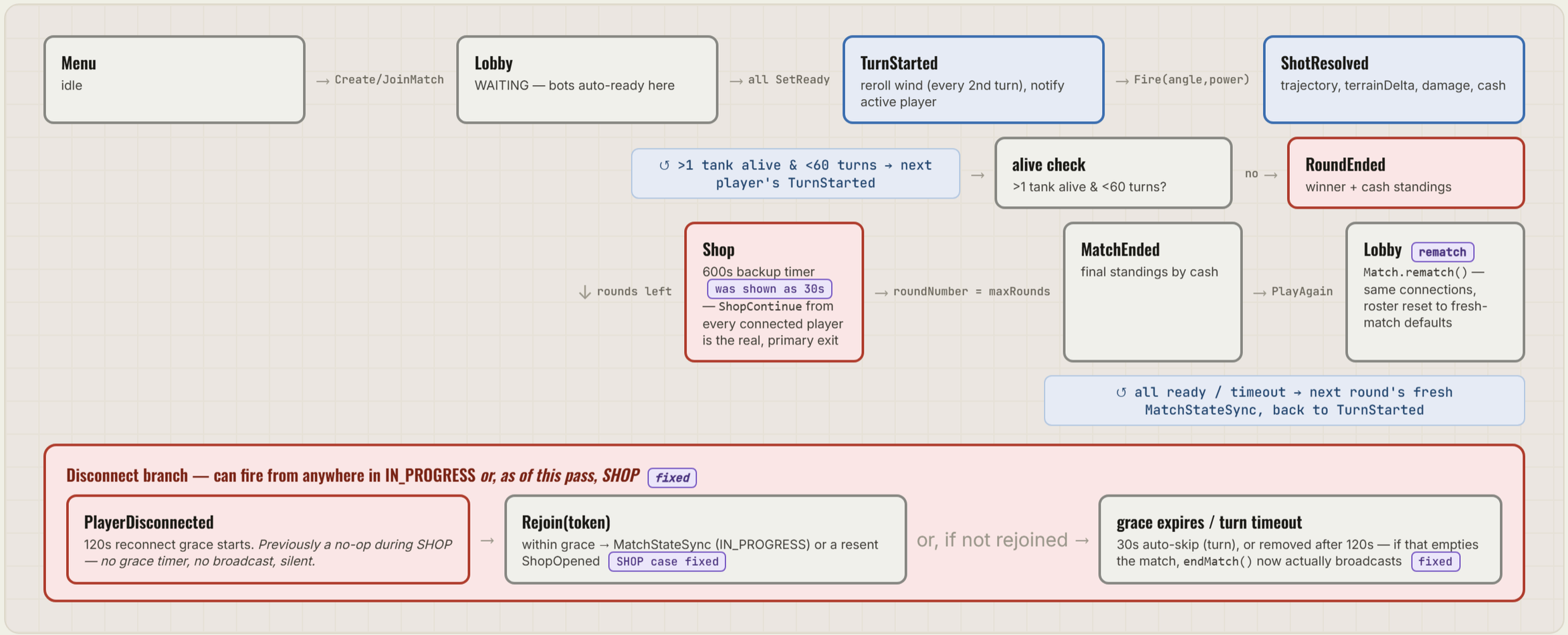

Fig. 5 — the blue path is the two nested cycles (turn-within-round, round-within-shop); the red path is the disconnect branch. Both round-trip through `Match`'s existing methods whether the actor is a human's WebSocket message or a bot's direct call (fig. 3).

FIG. 6 — NEW THIS PASS

WEAPON PHYSICS / RENDERING AUDIT

`ProjectileSim.simulate() swept, not guessed`

Live playtest with bots surfaced "the bot's shell has insane power" and "MIRV flies backwards after the split." Ran `ProjectileSim.simulate()` across every weapon × angle × power on flat terrain before touching any code.

The finding. Range scales ~power². At the old `POWER_SCALE=12`, a 45° shot already wrapped the 1600-unit cyclic map multiple times well before max power — and bots' grid search couldn't tell a direct hit from a wrapped one.

POWER @ 45°	OLD (12.0)	NEW (9.0)
50	1×	0×
75	2×	1×
100	4×	2×

1

Bot turret never aimed

`BotController` never called the existing `Match.updateAim`. Now does, ~500ms before firing.

2

MIRV flies backwards

2nd instance of the trajectory-wrap bug, this time in the fall-phase child solve. `shortestWrapDx`, shared with fix below.

3

Bots pick wrap-around shots

`BotAimPlanner` now analytically rejects any grid candidate that would require wrapping — no target ever needs one.

4

POWER_SCALE 12.0 → 9.0

A 25% cut for everyone — explicitly half the correction fully constraining bots represents, per the user's own framing.

5

Wind toned down

`WIND_ACCEL_PER_STRENGTH` 4.0 → 2.5 (confirmed linear, no curve). Cap scaled proportionally too.

6

Tunnel didn't follow the shot

New `Terrain.carveTunnelSegment` — max-based, path-following, replaces overlapping additive digs.

Before — pointAtProgress lerp across a wrap

After — shortestWrapDx

Fig. 6 — verified: full server+client suites green (new TerrainTest/BotAimPlannerTest/shortestWrapDx cases), plus a live WS-frame-level re-check confirming PlayerAiming now fires for bots and bot shots never wrap.

FIG. 7

FILED, NOT BUILT

Ideas the user has raised, written up in `PLAN.md`'s "Future ideas" section, not implemented. Refreshed this pass — the single-player-bot-opponent entry from the last read is gone (fig. 3), and three new ones landed the same session as the diagram work below.

Permanent cross-match leaderboard `new`

Top scorer per match tracked on an ongoing running total. Open: what "score" means, `playerId` is ephemeral/per-match today, and this would be the first genuinely persistent data anywhere in this project.

Wind scaled to match/bot difficulty `new`

Alternate to the flat wind tone-down (fig. 6). Distinct from bot skill difficulty — wind affects every player equally per turn, not per-bot. Needs its own scoping.

Connection/access log for troubleshooting `new`

Every join attempt (success/failure) logged — IP, user-agent, timestamp — low-resource flat text, date-linked entries, for troubleshooting under real load. Needs an event list, a rotation policy, and a privacy pass before it captures player IPs.

Digger: tunnel-then-collapse

The tunnel now visibly follows the trajectory (fig. 6) — the "then the sides collapse" half is still just whatever `Terrain.settleSlopes` already does generically, not a distinct Digger behavior.

Napalm damage-over-time / pooling

Should keep dealing damage while lingering, pooling deeper in terrain cavities. Genuinely new persistent turn-to-turn state — nothing in the engine carries state between turns today.

Heavy Cannonball rolls downhill after impact

Scorched Earth's "Roller" precedent — stops at a valley or a tank, detonates once at the end of the roll, not repeated bumps.

Directional (trajectory-shaped) blast falloff

Currently uniform radial falloff regardless of impact angle.

Sandhog / Tracer / Riot family / Leapfrog

New-weapon concepts surfaced by genre research (`docs/weapon-gap-analysis.md`), never explicitly requested — filed as candidates, not committed work.

Corrected from the last read, not re-listed above: "single-player + bot opponent" — built (fig. 3). "Bouncing Betty per-bounce damage, currently cosmetic only" — was already wrong; real per-bounce damage (`BOUNCE_DAMAGE_FRACTION`) has shipped since. "Per-weapon sound design, not built" — shipped in full (Web Audio synthesis, `docs/asset-sources.md`). "Only Nuke has distinct explosion art" — Napalm now has its own splash+flame effect too; still true that most of the roster shares one generic flash.

Source: PLAN.md · shared/protocol.md · client/src/** · server/src/main/java/com/brutaltank/** — read 2026-08-25, commit 1f54e78.